Lead Scraper Suite

LinkedIn Scraper & Google Scraper, powered by treg

Product Requirements Document (PRD)

Field	Value
Version	1.0 (draft for review)
Date	29 October 2026
Status	Awaiting owner review before build starts
Data layer	treg (github.com/superdesigndev/treg), hosted at treg.to
Deliverable	Two independent command-line scrapers that each output a lead CSV

1. Summary

We will build **two separate scrapers** that produce lead lists as CSV files:

- **LinkedIn Scraper**: finds companies on LinkedIn by keyword, industry, location and size (or takes a list of LinkedIn company URLs) and exports every public company field LinkedIn exposes, enriched with emails and phone numbers.
- **Google Scraper**: finds businesses through Google (Google Maps / Places listings and Google search results) for a query such as *"dentists in Austin, TX"* and exports name, phone, address, website, rating, hours, enriched with emails and social links.

Each scraper runs on its own. Running the LinkedIn scraper touches only LinkedIn data sources; running the Google scraper touches only Google data sources. They share a small common library (treg client, email/phone enrichment, CSV writer) so the code is written once, but neither depends on the other at run time.

Data is fetched through **treg**, an "OpenRouter for agent tools": one base URL and one token give access to a catalog of data providers (LinkedIn data APIs, Google search/maps APIs, email finders, phone finders), billed per call. This means we do **not** log in to LinkedIn or Google with our own accounts, run headless browsers, rotate proxies, or solve CAPTCHAs. treg holds the provider accounts and injects credentials server-side.

2. Goals and non-goals

2.1 Goals

- Produce a clean, de-duplicated **lead CSV** per run, ready to import into a CRM or Google Sheets.
- Capture **company emails**, **contact numbers**, and **every public detail** available on the source (LinkedIn company page or Google business listing).
- Keep LinkedIn and Google scraping **fully separate**: separate commands, separate CSVs, separate configs.
- Control cost: show an estimate before a run, enforce a hard budget cap, and report actual spend after a run.
- Be resumable: a crashed or stopped run continues where it left off without paying twice for the same lead.

2.2 Non-goals (version 1)

- No web dashboard or GUI. Command line only. (Can be added later.)
- No sending of emails / outreach automation. The tool only builds lists.
- No direct CRM sync (HubSpot, Salesforce). CSV is the integration point.
- No scraping by logging in to personal LinkedIn or Google accounts, no browser automation, no CAPTCHA bypass.
- No scheduling / recurring monitoring. Can be added later with cron or Claude Code /schedule.
- No merged "LinkedIn + Google" mode. Both CSVs share the same first columns, so they can be merged later with one small script if needed.

3. Users and use cases

#	As a...	I want to...	So that...
U1	Sales / agency owner	pull all software companies in Germany with 11-50 staff from LinkedIn	I get a targeted B2B prospect list with emails
U2	Local-service marketer	pull all "roofing contractors in Dallas" from Google Maps	I get phone numbers, websites and emails of local businesses
U3	Researcher	paste a list of 200 LinkedIn company URLs	I get full company details for exactly those companies

#	As a...	I want to...	So that...
U4	Any user	see the expected cost before scraping and cap spend	I never get a surprise bill
U5	Any user	re-run a job after an interruption	already-scraped leads are skipped and not paid for again

4. How treg is used

treg (Apache 2.0 with additional terms) is a registry and proxy. Its license allows calling the hosted treg.to API from our own product; what it forbids is re-selling treg itself as a competing hosted registry, which we are not doing.

4.1 Mechanics

- **Auth:** every request sends header `X-Treg-Token: <token>` (team token; a personal token also needs `X-Treg-Org`).
- **Call:** `GET https://treg.to/call/<endpoint_id>?key=value` (or POST with JSON body). The provider's response comes back verbatim.
- **Cost:** each response carries `X-Treg-Cost-Micro` (micro-USD). `X-Treg-Route-Max-Cost` caps a single call. New teams get $1.00 free credit.
- **Errors:** HTTP 402 = out of balance (stop the run, save progress); 503 = provider account empty (not charged, skip/retry later); 502 = response too large (not charged).
- **Routed endpoints** (`treg.*`) let treg pick the best provider, e.g. `treg.people.email.find`, `treg.people.email.verify`, `treg.people.phone.find`, `treg.people.search`.

4.2 Endpoints we need (capabilities)

treg's catalog is served from treg.to and changes over time; the README does not publish fixed IDs for every LinkedIn or Google endpoint. Phase 0 (section 10) therefore finds the current best endpoint for each capability with `treg catalog search` and records its ID, price and hit rate in a config file. Swapping a vendor later is a one-line config change.

Capability	Used by	How to find it / known ID
LinkedIn company search (keyword, industry, location, size)	LinkedIn	`treg catalog search "linkedin company search"`
LinkedIn company profile (all public fields)	LinkedIn	`treg catalog search "linkedin company profile"`
LinkedIn people at a company (optional decision makers)	LinkedIn	`treg.people.search` (routed)
Google Maps / Places search (query + location)	Google	`treg catalog search "google maps places search"`
Google Place details (hours, reviews count, etc.)	Google	`treg catalog search "google place details"`
Google web search results (SERP)	Google	routed Google-results capability, or `treg catalog search "google search results"`
Web page extract (JS-heavy websites)	Both (enrichment)	routed scrape-a-page capability, e.g. `treg catalog search "extract web page"`
Email by domain / by person	Both (enrichment)	`hunter.people.email.find`, `treg.people.email.find`
Email verification	Both (optional)	`treg.people.email.verify`
Phone finder (person)	LinkedIn (optional)	`treg.people.phone.find`

5. Functional requirements

5.1 LinkedIn Scraper

ID	Requirement	Priority
L1	Search companies by keyword, industry, location (country/city) and company size range.	Must
L2	Alternative input: a text/CSV file of LinkedIn company URLs (one per line).	Must
L3	For every company, fetch the full public company profile and map all fields in section 6.1.	Must
L4	Find company emails via the enrichment pipeline (section 5.3).	Must
L5	Capture phone from the LinkedIn page when present; otherwise from the company website.	Must
L6	Optional `--decision-makers "CEO,Founder,Head of Sales"`: find up to N people per company with name, title, LinkedIn URL, work email (and phone if `--person-phone`).	Should
L7	`--limit` on the number of companies.	Must
L8	De-duplicate by LinkedIn company URL / ID.	Must

5.2 Google Scraper

ID	Requirement	Priority
G1	Search Google Maps / Places by query + location (e.g. "plumbers", "Austin, TX"), paging until `--limit` or results run out.	Must
G2	Fetch place details for each result and map all fields in section 6.2.	Must
G3	Alternative `--mode serp`: use Google web search results (organic listings) instead of Maps, collecting title, URL, snippet, then enriching the domain.	Should
G4	Multiple locations in one run: `--locations-file cities.txt`.	Should
G5	Find emails and social links (Facebook, Instagram, LinkedIn, X, YouTube) via enrichment (5.3).	Must
G6	De-duplicate by Google place_id (fallback: normalized phone + domain).	Must
G7	Optional filters: minimum rating, minimum review count, skip permanently closed.	Should

5.3 Shared enrichment pipeline (emails, phones, socials)

Neither LinkedIn company pages nor Google listings normally show an email address, so emails come from the company's own website and from email-finder providers. The pipeline runs cheapest-first and stops when it has enough:

Step	Source	Cost	Output
1	Fetch the company website homepage plus /contact, /contact-us, /about, /impressum directly over HTTP; parse mailto: links, visible emails, tel: links, social links.	Free	emails, phones, socials
2	If the site is JavaScript-rendered and step 1 found nothing, fetch the same pages through treg's page-extract capability.	Paid (small)	emails, phones, socials
3	If still no email, query an email finder by domain (e.g. hunter via treg).	Paid	generic + named emails with confidence
4	(Optional, `--verify`) verify each email; mark valid / risky / invalid.	Paid	email_status
5	(Optional, LinkedIn decision makers) `treg.people.email.find` with the person's LinkedIn URL; `treg.people.phone.find` if requested.	Paid	person email / phone

- Emails are filtered: drop image-file false positives (e.g. *logo@2x.png*), placeholder addresses (example.com, sentry, wixpress) and addresses on unrelated domains unless they are the only one found.
- Phones are normalized to E.164 (e.g. +15125550100) using the lead's country.
- Every email and phone records its **source** (linkedin, google, website, hunter, ...).

5.4 Common behaviour (both scrapers)

ID	Requirement	Priority
C1	`--dry-run`: look up endpoint prices via the catalog and print an estimated cost for the requested limit, then exit without spending.	Must
C2	`--max-cost USD`: hard budget; stop cleanly when cumulative X-Treg-Cost-Micro reaches it. Also sends X-Treg-Route-Max-Cost per call.	Must
C3	Write the CSV row-by-row as each lead completes (no data loss on crash).	Must
C4	`--resume`: read the existing CSV, skip leads already present.	Must
C5	`--no-enrich`: skip the email/phone pipeline (cheaper, source data only).	Must
C6	Retry on 429 / 5xx with exponential backoff (max 3 tries); stop the run on 402 and print the top-up URL.	Must
C7	End-of-run summary: leads written, % with email, % with phone, total cost, cost per lead, errors.	Must
C8	Keep a JSONL run log of each treg call (endpoint, status, cost, served_by) for debugging and audit.	Should
C9	Suppression list: `--suppress suppress.txt` of domains/emails never to output.	Should

6. Output: lead CSV format

UTF-8 with BOM (opens correctly in Excel), comma-separated, one row per company. Multi-value fields (several emails, several phones) are joined with " | ". The **first 12 columns are identical in both scrapers** so the two CSVs can be stacked into one sheet later.

Common columns (both CSVs, in this order)

Column	Description
company_name	Business / company name
website	Company website URL
domain	Normalized domain, e.g. acme.com
email	Best single email (verified > named > generic info@)
all_emails	Every email found, " | " separated
email_source	Where the best email came from: website / hunter / linkedin / ...
phone	Best phone in E.164 format
all_phones	Every phone found
address	Full address as given by the source
city / country	Parsed city and country (two separate columns)
source_url	LinkedIn company URL or Google Maps URL of the record

6.1 LinkedIn CSV: additional columns

Column	Description
linkedin_id	LinkedIn numeric company ID / universal name
tagline, description	Company tagline and About text
industry	LinkedIn industry
company_size	Size band, e.g. 11-50
employees_on_linkedin	Count of associated members
headquarters, all_locations	HQ and every office address listed
founded_year, company_type	e.g. 2014, Privately Held
specialties	Specialties list

Column	Description
followers	LinkedIn follower count
logo_url, cover_url	Images
funding_info	Last round / investors when the provider returns it
socials	Facebook, Instagram, X, YouTube links found on the website
decision_makers	(optional) JSON list: name, title, linkedin_url, email, phone
email_status	(optional) valid / risky / invalid
scraped_at, cost_usd	Timestamp and treg cost spent on this row

6.2 Google CSV: additional columns

Column	Description
place_id	Google place ID (dedupe key)
category, all_categories	Primary and secondary business categories
state, postal_code, latitude, longitude	Location details
rating, review_count	Google rating and number of reviews
price_level	When available
opening_hours	Weekly hours, " | " separated by day
business_status	OPERATIONAL / CLOSED_TEMPORARILY / CLOSED_PERMANENTLY
google_maps_url	Direct Maps link
facebook, instagram, linkedin, x, youtube	Social links from the website
search_query, search_location	The query that produced this lead
serp_rank, serp_snippet	(SERP mode only) position and snippet
email_status, scraped_at, cost_usd	As in LinkedIn CSV

7. Command-line interface

```
# one-time setup
pip install -r requirements.txt
set TREG_TOKEN=...          (Windows)  /  export TREG_TOKEN=...  (macOS/Linux)

# LinkedIn
python linkedin_scraper.py --keyword "SaaS" --location "Germany" --size 11-50 \
       --limit 200 --max-cost 10 --out leads_linkedin.csv
python linkedin_scraper.py --input company_urls.txt --decision-makers "CEO,Founder" --verify
python linkedin_scraper.py --keyword "fintech" --location "London" --limit 50 --dry-run

# Google
python google_scraper.py --query "dentists" --location "Austin, TX" \
       --limit 300 --max-cost 5 --out leads_google.csv
python google_scraper.py --query "roofing contractor" --locations-file cities.txt --min-rating 4
python google_scraper.py --query "web design agency Dubai" --mode serp --limit 100

# common flags: --dry-run --max-cost --resume --no-enrich --verify --suppress FILE --out FILE
```

8. Technical design

8.1 Stack

- Python 3.11+. Dependencies kept to a minimum: **httpx** (HTTP to treg and websites) and **phonenumbers** (E.164 normalization). CSV, argparse, JSON, regex and HTML parsing from the standard library.

- treg CLI used only by the developer for setup and endpoint discovery (`treg login`, `treg catalog search/get`); the scrapers themselves call treg over plain HTTPS.
- Runs on Windows, macOS and Linux. No database, no server, no browser.

8.2 Project layout

```
lead_scraper/
  linkedin_scraper.py   # CLI: LinkedIn search -> profile -> enrich -> CSV
  google_scraper.py     # CLI: Google Maps/SERP search -> details -> enrich -> CSV
  common.py             # treg call() with retry + cost/budget tracking,
                        # website email/phone/social extractor, CSV writer/resume
  endpoints.json        # endpoint IDs per capability (from Phase 0), editable
  requirements.txt      # httpx, phonenumbers
  .env.example          # TREG_TOKEN=
  test_common.py        # small checks for email/phone parsing and CSV resume
```

8.3 Data flow

```
 LinkedIn:  input (search or URL list)
              -> treg: linkedin company search  -> company URLs (dedupe)
              -> treg: linkedin company profile -> raw fields
              -> enrich(website)  [free crawl -> treg extract -> email finder -> verify]
              -> (optional) treg.people.search + people.email.find
              -> append row to leads_linkedin.csv

 Google:    query x location(s)
              -> treg: google maps search (paged) -> place_ids (dedupe)
              -> treg: place details            -> raw fields
              -> enrich(website)  [same pipeline]
              -> append row to leads_google.csv
```

8.4 Key rules

- One `call(endpoint, params)` function in common.py is the only place that talks to treg: it adds the token, enforces the budget, retries, logs, and returns JSON.
- Field mapping from provider JSON to CSV columns lives in each scraper, in one function, so a vendor change only touches that function and endpoints.json.
- The token is read from the environment, never written to CSV, logs or code.
- Concurrency: a small thread pool (default 4) for website crawling; treg calls sequential at first, raised only if speed is a problem.

9. Non-functional requirements

Area	Requirement
Cost	No run can exceed --max-cost. Cost per lead printed at the end. Website crawling (free) always tried before paid email finders.
Reliability	Rows are flushed after each lead; a crash loses at most the lead in progress. --resume never pays twice for a lead already in the CSV.
Performance	Target: 100 Google leads with enrichment in under 10 minutes; 100 LinkedIn companies in under 15 minutes (to be confirmed in Phase 0 against real provider latency).
Data quality	No duplicate rows; emails syntactically valid and de-junked; phones in E.164; empty fields left empty (never "N/A").
Security	TREG_TOKEN only in environment / .env (git-ignored). No credentials for LinkedIn or Google are stored anywhere in this project.
Portability	Windows-first (owner's machine), also works on macOS/Linux.

10. Milestones

Phase	Scope	Done when
0. Setup & discovery	Install treg CLI, `treg login`, create team token. Run `treg catalog search` for every capability in 4.2; `treg catalog get` for price; one test call each; fill endpoints.json.	Every capability has a chosen endpoint ID, price and a saved sample response.
1. Common core	common.py: treg call + budget + retry + log; website extractor; CSV writer + resume; tests.	Tests pass; one manual call through call() works.
2. Google Scraper	Maps search, details, enrichment, CSV, dry-run, summary. SERP mode after Maps works.	Acceptance tests G-A1..G-A3 pass.
3. LinkedIn Scraper	Company search + URL-list input, profile mapping, enrichment, CSV.	Acceptance tests L-A1..L-A3 pass.
4. Optional extras	Decision makers, email verification, suppression list, multi-location.	Each flag works on a 10-lead sample.
5. Hardening	Real runs of 200+ leads each, fix edge cases, README with examples.	Owner signs off on two real CSVs.

Google is built before LinkedIn on purpose: Maps listings already include phone and website, so the shared enrichment pipeline gets tested on the easier source first.

11. Acceptance criteria

ID	Test	Pass condition
G-A1	google_scraper.py --query "dentists" --location "Austin, TX" --limit 50	50 unique rows; 90%+ have phone; 80%+ have website; 40%+ have an email (target, check in Phase 0)
G-A2	Same command with --max-cost 0.10	Stops at or below $0.10, CSV contains the rows paid for
G-A3	Kill the run mid-way, rerun with --resume	No duplicates; no repeated paid calls for finished leads
L-A1	linkedin_scraper.py --keyword "SaaS" --location "Germany" --limit 50	50 unique companies with name, LinkedIn URL, industry, size, HQ, website for 90%+
L-A2	linkedin_scraper.py --input 20_urls.txt	Exactly those 20 companies, all section 6.1 fields the provider returns
L-A3	Any run with --dry-run	Prints estimate, makes zero paid calls (balance unchanged)
X-A1	Open both CSVs in Excel	Opens with correct characters; first 12 columns identical in name and order

Email hit rate is the main unknown. Many small businesses list no email on their website and are absent from email-finder databases. 40-60% email coverage is typical for local businesses; B2B tech companies are usually higher. Phase 0 measures the real number on 20 sample leads before we commit to targets.

12. Cost model

treg bills per call in micro-USD; your own API keys added to treg are free. Real prices come from `treg catalog get <id>` in Phase 0. The formula the dry-run uses:

```
cost_per_lead = search_cost / results_per_page
              + details_cost
              + p(needs_extract) * extract_cost
              + p(no_email_after_crawl) * email_finder_cost
              + (verify ? emails_per_lead * verify_cost : 0)
              + (decision_makers ? N * (people_search_share + email_find_cost) : 0)
```

The README describes most lookups as "fractions of a cent per call"; email finding is usually the most expensive step, which is why it runs last and only when the free website crawl found nothing. The $1.00 free credit on a new treg team is enough for Phase 0 testing.

13. Legal and compliance

- **LinkedIn and Google terms of service** restrict automated scraping. This design does not scrape them directly; it uses third-party data providers through treg. The owner is still responsible for how the data is used and should review each provider's terms (shown in `treg catalog get`).
- **GDPR / UK GDPR:** contacting people in the EU/UK needs a lawful basis (usually legitimate interest, documented). Named personal emails (john@company.com) are personal data; generic ones (info@) are lower risk.
- **CAN-SPAM / similar laws:** outreach must include identification and an unsubscribe option.
- **Suppression list** (C9) is applied before anything is written, so opted-out contacts never re-appear.
- Data is stored only in local CSV files on the owner's machine.

14. Risks

Risk	Impact	Mitigation
A needed LinkedIn or Google endpoint is missing from treg's catalog or gets removed	High	Phase 0 checks before coding; endpoints.json makes vendors swappable; `treg catalog request` to ask for one; treg also lets us add our own API key for a provider (e.g. a Google Places key) as a private tool
Low email coverage	Medium	Cheapest-first pipeline, --verify, measured in Phase 0, honest coverage in run summary
Price changes / runaway spend	Medium	--dry-run, --max-cost, per-call Route-Max-Cost header
Provider returns different JSON shapes	Low	One mapping function per scraper; sample responses saved in Phase 0
Rate limits / provider outages (429, 503)	Low	Backoff + retry; 503 is not charged; resume

15. Open questions for the owner

Please answer these before Phase 1; the Default column is what we will build if no answer is given.

#	Question	Default
Q1	Which countries / cities will you target first?	US + one EU country for testing
Q2	Do you need decision-maker people (CEO, founder) with personal work emails, or only company-level emails and phones?	Company-level in v1; decision makers as optional flag
Q3	Typical run size and budget per run?	200 leads, $5 cap
Q4	Should emails be verified by default (extra cost)?	Off; enable with --verify
Q5	Use the hosted treg.to service or self-host treg?	Hosted treg.to
Q6	Do you already have API keys for any provider (Google Places, Hunter, Apollo, etc.)? They can be added to treg and used at no treg charge.	None
Q7	Is a command-line tool fine, or do you want a simple web UI later?	Command line
Q8	Excel (.xlsx) output in addition to CSV?	CSV only

Next step: owner reviews this document and answers section 15; then we start Phase 0 (treg login and endpoint discovery) together.